

Informatik BW: Python Crash Course B

VU Informatik BW, WS24

Thorsten Ruprechter

- 👉 Variablen
- 👉 Integers, Floats, Bool'sche Werte
- 👉 Operatoren
- 👉 Strings
- 👉 Casting
- 👉 None-Keyword
- 👉 Assignment 0

Kurze Wiederholung: Variablen und Datentypen

 feedback
fbt.io/info/bw2

Zeit: 6 Minuten



- 👉 Besprechung feedbackr.io
- 👉 Bedingungen (`if`-, `if-else`-, `if-elif-else`-Statements)
- 👉 Lists
- 👉 Tuples
- 👉 `for`-Loops

Verzweigungen (0): if Statements

☞ Überprüfen von Bedingungen, konditionelles Ausführen von Code

- „WENN-Bedingung“ / “WENN-DANN-Bedingung“

☞ „**WENN** *Bedingung erfüllt* **DANN** *tu etwas*“

☞ Python Syntax:

```

condition:
    print('Do something')
4 Leerzeichen eingerückt
    
```

```

if x > 2:
    print('In if')
    print('Not in if')
    
```

Verzweigungen (1): if-else Statements

☞ if-Statement mit „Fallback“ falls Bedingung nicht erfüllt

- „WENN-DANN-SONST-Bedingung“

☞ „**WENN** *Bedingung erfüllt* **DANN** *tu etwas* **SONST** *tu etwas anderes*“

☞ Python Syntax:

☞ **if** *condition*:
 print('Do something')

☞ **else**:
 print('Do something else')

```
if x > 2:  
    print('In if')  
else:  
    print('In else')
```

Verzweigungen (2): if-elif-else Statements

☞ Mehrere Verzweigungen für kompliziertere Abfragen

• „WENN-DANN-SONSTWENN-DANN-SONSTWENN-...“

☞ **WENN** *Bedingung1* erfüllt **DANN** *tu etwas*

☞ **SONST WENN** *Bedingung2* erfüllt **DANN** *tu etwas anderes*

☞ **SONST WENN** *Bedingung3* erfüllt **DANN** *tu wieder etwas anderes*

☞ ...

☞ **SONST** *tu etwas völlig anderes*

Verzweigungen (3): if-elif-else Statements (Syntax)

```
if condition:
    print('Do something')
elif condition2:
    print('Do something else')
elif condition3:
    print('Do something different')
    # there could be more elifs below this
else: # optional

    • print('Do something completely different')
```


Verzweigungen (4): if-elif-else Statements (Beispiel)

≠

```
# Snippet A
if x < 5:
    print('Small')
elif x < 15:
    print('Medium')
elif x < 20:
    print('Big')
•else:
    print('Huge')
```

```
# Snippet B
if x < 5:
    print('Small')
if x < 15:
    print('Medium')
if x < 20:
    print('Big')
•else:
    print('Huge')
```

≠

Einrückungen: Indentation rules

🔗 Standard: PEP8 (<https://www.python.org/dev/peps/pep-0008/>)

- = 4 Leerzeichen, **KEINE TABS!** (siehe Editor-Config)

🔗 Block-Einrückungen und -Anweisungen:

- „Alles was nach **:** eingerückt ist, gehört zu diesem Block“

– Ähnlich: Einrückungen im Powerpoint!

- Sprach-/Syntax-Element

– Missachtung = Error

– Implizit: bessere Formatierung von Code durch Syntax

- Ändert Verhältnis zu restlichem Programm
- if, for, while, def, ...

🔗 !!! Indentation Errors

```
if x > 2:  
    print('Do something')
```

↑
4 Leerzeichen eingerückt

Einrückungen: Schachtelung

👉 „Code-Blöcke“ beliebig schachtelbar

- Beispiel mit `if-else`:

 = 4 Leerzeichen eingerückt

```
age_max = 35
age_peter = 42
age_berta = 37
if age_max < age_peter:
    print('Max is younger than Peter')
     if age_berta < age_max:
        print('Berta is the youngest!')
     else:
        print('Max is the youngest!')
     if age_berta < age_peter:
        print('Berta is the youngest!')
     else:
        print('Peter is the youngest!')
```

Verzweigungen (6): Beispiele

Siehe Code-Beispiele

Lists (0): Einführung

☞ Sammlung von Elementen – **geordnet** und **veränderbar**

- Typ von Elementen egal

☞ `[]` erzeugt Liste, Elemente getrennt durch Beistrich

- `colors = ['blue', 'red', 'green']`

- `empty_list = []`

☞ Zugriff auf Elemente durch `[]` und index – **0 basiert !**

- `colors[1] => 'red'`

- Indizieren von hinten: negativer Index (-1 = letztes Element)

Lists (1): Elemente bearbeiten und hinzufügen

🔗 Bearbeiten

- „Element-Zugriff mit Variablenzuweisung“

```
-colors[0] = 'grey'
```

🔗 Einfügen

- Am Ende: `append()`

```
-colors.append('orange')
```

- Beliebige Stelle: `insert()`

```
-colors.insert(1, 'yellow')
```

Lists (2): Elemente löschen und Listen-Länge

🔗Löschen

- Einfaches entfernen: `del` mit Index

```
-del colors[2]
```

- Löschen und Speichern: `pop()`, mit oder ohne (= letzter) Index

```
-removed_color = colors.pop()
```

```
-removed_color = colors.pop(1)
```

- Löschen mittels Wert: `remove()`

```
-colors.remove('grey')
```

🔗Länge von Listen: Funktion `len()`

Lists (3): IndexError

👉 **Python Sequenzen starten mit 0 !** (siehe Strings)

```
numbers = [1, 13, 42]
```

```
print(numbers[3])
```

=> „IndexError: list index out of range“

👉 Indizieren mit `[-1]` = sicher letztes Element (siehe String-Slicing!)

- IndexError wenn Liste leer
- Generell: Achtung mit Indizierung!

Lists (4): Slicing

☞ Siehe Strings - Exakt selbe Syntax!

• Warum? – Strings $\approx$ Sequenz/Liste von Zeichen

☞ Zugriffsarten (Auswahl – siehe Slides 21 bis 23 letzte Einheit)

• `items[n]`

– n-tes Zeichen

• `items[start:ende]`

– Zeichen von start (inklusive) bis ende (exklusive)

• usw.

Lists (5): Zugehörigkeitsoperatoren

Operator	Beschreibung	Beispiel
<code>in</code>	Wahr, wenn Element (oder Sequenz) in Objekt ist	<code>1 in number_list</code> <code>[1, 2] in number_list</code> <code>'HELP' in string</code>
<code>not in</code>	Wahr, wenn Element (oder Sequenz) nicht in Objekt ist	<code>1 not in number_list</code> <code>['A', 'B'] not in letter_list</code> <code>'HELP' not in 'Hello World'</code>

Tabelle 1: Zugehörigkeitsoperatoren

Lists (6): Sortieren und Ordnen

🔑 Sortieren

- Permanent: Methode `sort()`

- `colors.sort()`

- Temporär (gibt sortierte Liste zurück): Funktion `sorted()`

- `sorted_colors = sorted(colors)`

- Strings: Achtung bei Groß-/Kleinschreibung

🔑 Reversieren: Methode `reverse()` und Funktion `reversed()`

- `colors.reverse()`

- `reversed_colors = reversed(colors)`

- Äquivalent: `colors[::-1]`

Lists (7): Numerische Listen

🔑 Generieren von Zahlensequenzen (NICHT Listen): `range()`

- Parameter (ähnlich Slicing):

- Start (optional, Default-Wert 0), Ende, Intervall (optional)

- inklusive Start, exklusive End

🔑 Beispiele:

- `range(4)`

=> 0, 1, 2, 3

- `range(1, 4)`

=> 1, 2, 3

- `range(1, 10, 3)`

=> 1, 4, 7

🔑 Umwandlung von `range`-Objekt in Liste: `list(range())`

Lists (8): Statistiken mit Listen

```
num_list = list(range(3))
```

=> [0, 1, 2]

👉 Minimum: `min(num_list)`

=> 0

👉 Maximum: `max(num_list)`

=> 2

👉 Summe: `sum(num_list)`

=> 3

=> 1.0

👉 Durchschnitt: `sum(num_list)/len(num_list)`

Tuples

👉 **Geordnet** und **unveränderbar**

👉 Definition durch `()`, Elemente getrennt durch `,`

👉 Elementzugriff per Index

- `colors[1]`

👉 Geringer interner Overhead

👉 Beispiele für Verwendung:

- Zusammengehörige Werte

- Fixe Sequenzen

👉 `len()`

```
base_colors = ('red', 'blue', 'green')
```

Lists: Beispiele

Siehe Code-Beispiele

for-Loops (0): Basics von for-Schleifen

☞ Iterieren/Durchlaufen von Sequenzen (z. B. Listen)

☞ **FÜR** jedes Element **IN** Sequenz:

☞ Führe Anweisungen aus

☞ **Python-Syntax für for-Loops (Schleifen):**

```
for item in list_of_items:  
    print('This will be repeated.')
```

☞ Laufvariablen können beliebigen Namen haben

☞ Wiederholter Code-Block mit beliebig vielen Anweisungen

for-Loops (1): Beispiel

```
lectures = ['Maths', 'English', 'Computer Science']  
  
for lecture in lectures:  
    • print(f'I love {lecture}')
```

for-Loops (2): Beispiel

```
lectures = ['Maths', 'English', 'Computer Science']
```

```
for lecture in lectures:  
    print(f'I love {lecture}')
```

Legende

Iterierte Sequenz (z. B. Liste)

Definition der Sequenz

for-Loop Definition (Syntax)

Wiederholter Code Block

Lauf-/Schleifen-Variable

for-Loops (3): Iterieren über Strings („Buchstabieren“)

```
string = 'Informatik BW'
```

```
for char in string:  
    print(char)
```

for-Loops (4): Zählen mit `range()`

```
print('Count from one to ten!')  
for number in range(1, 11):  
    print(number)
```

for-Loops (5): Index mit Elementen ausgeben mit `enumerate`

```
lectures = ['Maths', 'English', 'Computer Science']  
  
for index, lecture in enumerate(lectures):  
    print(f'Lecture {index}: {lecture}')
```

for-Loops (6):

Mehrere Listen auf einmal durchlaufen mit `zip`

```
lectures = ['Maths', 'English', 'Computer Science']  
teachers = ['Browne', 'Orwell', 'Turing']
```

```
for lecture, teacher in zip(lectures, teachers):  
    print(f'Lecture {lecture}held by {teacher}')
```

Loops: Keywords zum Schleifen- und Programmablauf

 `pass`

- „leerer“ Command, tut nichts

 `break`

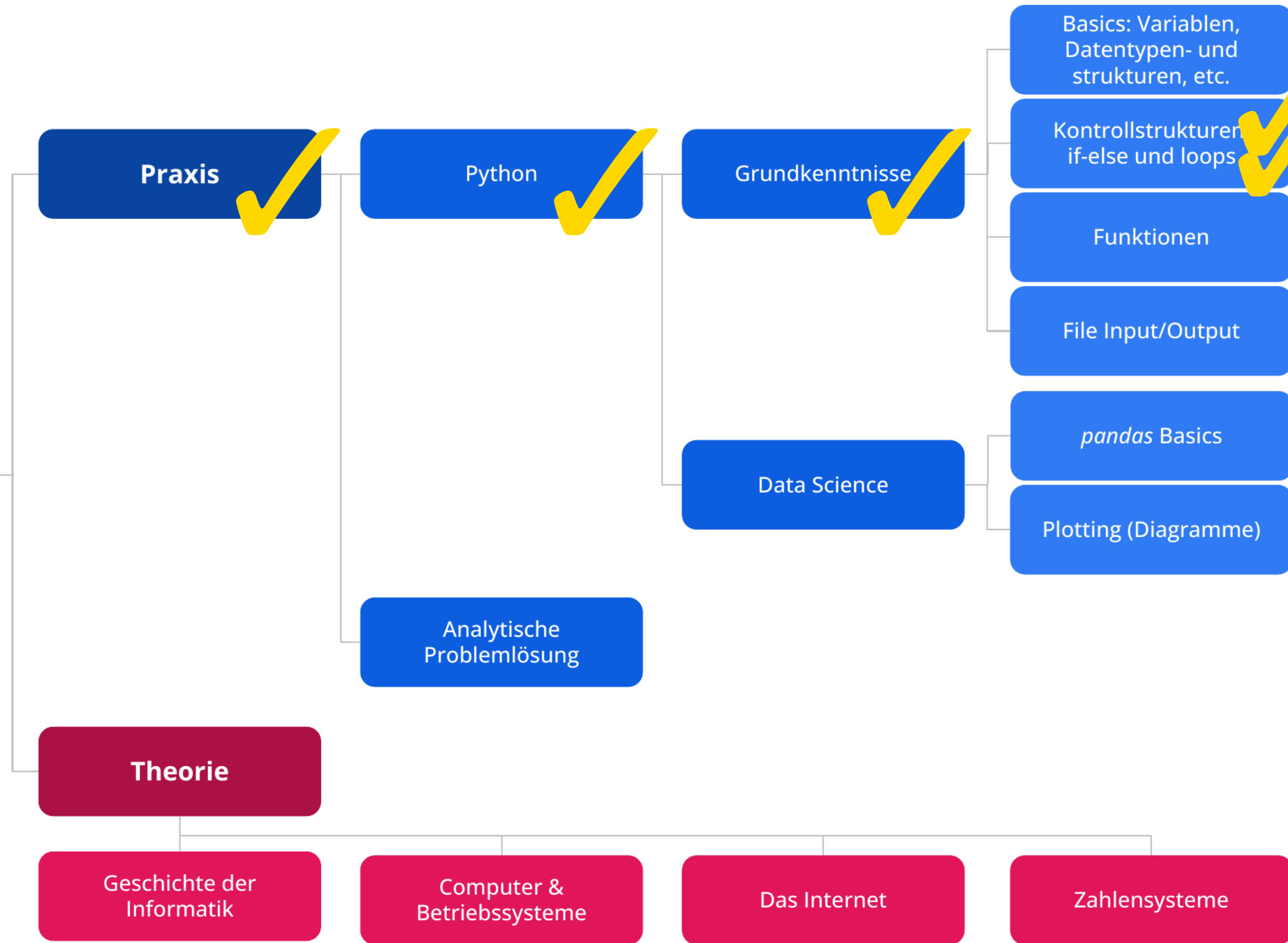
- Schleifenabbruch
- Sofortiges Beenden der „innersten Schleife“

 `continue`

- Sprung zu nächster Iteration
- Abbruch aktueller Durchlauf „innerster Schleife“

for-Loops (3): Basic Beispiele

Siehe Code-Beispiele



Informatik BW: Python Crash Course B

VU Informatik BW, WS24

Thorsten Ruprechter